

이제 충분한 데이터를 확보했습니다. 핵심을 잡겠습니다: “우리만 보여줄 수 있는 것” = 국가별 BTR/NDC 이행 추적 + 배출 격차 분석 + 적응-취약성-재원 통합 뷰. 이것을 기술적으로 어떻게 구현하는지를 중심으로, 순수 기술 문서 45-50페이지를 작성합니다.

VISUAL CLIMATE — 기술 사양서

VC-TECH-SPEC-2026-001 · v 3.0 · 2026-09-07
문서 유형: 기술 개발 사양서 (Technical Development Specification)

목차

Part 1	시스템 아키텍처	1
Part 2	데이터베이스 설계	5
Part 3	데이터 수집 파이프라인 (ETL)	13
Part 4	백엔드 API	22
Part 5	핵심 분석 엔진	28
Part 6	프론트엔드 & 시각화	34
Part 7	PWA · i18n · 접근성	40
Part 8	보안 · 테스트 · 배포	43
부록 A	전체 데이터 소스 카탈로그 (40개)	47

Part 1 — 시스템 아키텍처

1.1 차별화 전략: 왜 Visual Climate인가

현재 기후 데이터 플랫폼들은 각각 한 가지 축만 다룬다. Climate Watch는 배출+NDC, Our World in Data는 배출+에너지 시계열, ND-GAIN은 취약성만, CPI는 재원만 제공한다. 어떤 플랫폼도 한 국가에 대해 다음 질문에 한 화면으로 답하지 못한다:

“이 나라의 배출은 어디서 나오고(배출 프로파일), NDC 목표를 달성하고 있는지(이행 격차), BTR을 제출했는지(투명성), 기후 변화에 얼마나 취약하고(취약성), 적응 계획은 있는지(NAP), 기후 재원은 얼마나 받고 있는지(재원), 재해 피해는 어떤지(재해)”

Visual Climate는 이 7개 축을 하나의 **국가 기후 대시보드(Country Climate Dashboard)**에 통합한다. 각 수치는 복수 소스를 병렬 표시하여 투명성을 보장하고, NDC 이행 격차는 실제 배출 궤적과 목표를 겹쳐서 시각화한다.

1.2 핵심 분석 기능 (다른 플랫폼에 없는 것)

기능 1 — NDC 이행 격차 분석 (NDC Gap Tracker)

국가의 실제 배출 궤적(EDGAR/Climate TRACE)을 NDC 목표(Climate Watch NDC Content API)와 겹쳐서 "현재 추세대로라면 목표 달성 가능한가?"를 시각화한다. 선형 외삽(linear extrapolation)과 AR6 시나리오 경로를 함께 표시한다.

기능 2 — BTR 투명성 스코어보드

UNFCCC BTR 제출 현황(101개국 제출 완료, 2025 synthesis report 기준)을 추적한다. 각 국가의 BTR 구성 요소(NIR, CRT, CTF, 적응, REDD+) 제출 여부를 체크리스트로 표시하고, Common Reporting Tables의 주요 수치를 구조화한다.

기능 3 — 적응-취약성-재원 삼각 분석

ND-GAIN 취약성 지수 × INFORM 위험 지수 × GCF/CPI 기후 재원 수령액을 하나의 산점도(scatter plot)에 매핑하여 "가장 취약하지만 재원을 가장 적게 받는 국가"를 식별한다.

기능 4 — 섹터별 감축 잠재량 매핑

Climate TRACE의 자산 수준(asset-level) 데이터와 Top Emission Reduction Strategies 데이터셋을 결합하여 국가별·섹터별 감축 잠재량을 정량화한다.

기능 5 — 기후 전망 통합 뷰

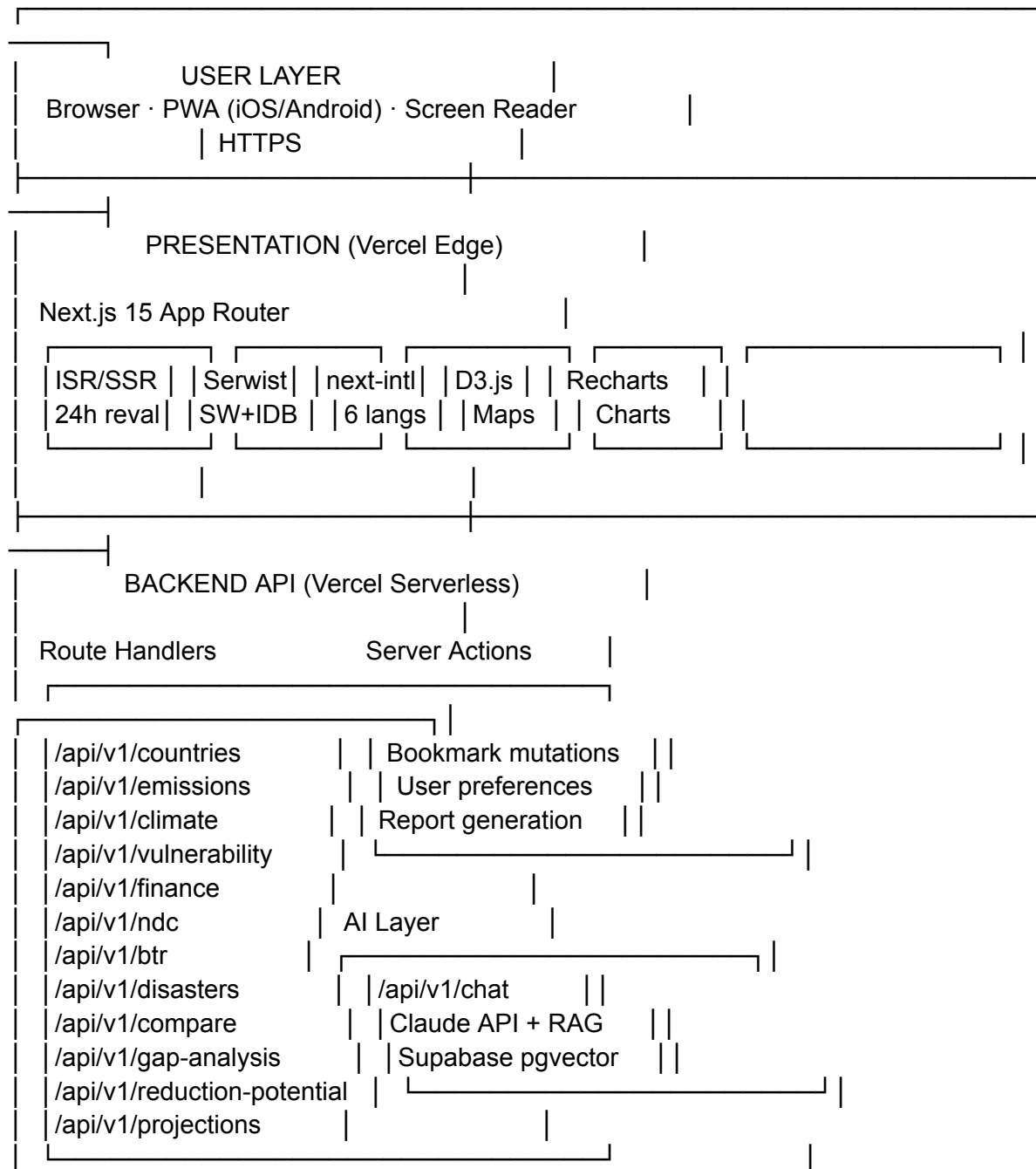
CCKP(World Bank Climate Change Knowledge Portal)의 CMIP6 전망 데이터(SSP1-2.6 ~ SSP5-8.5)를 국가별로 보여주고, 이를 해당 국가의 배출 궤적 및 NDC 목표와 연계한다.

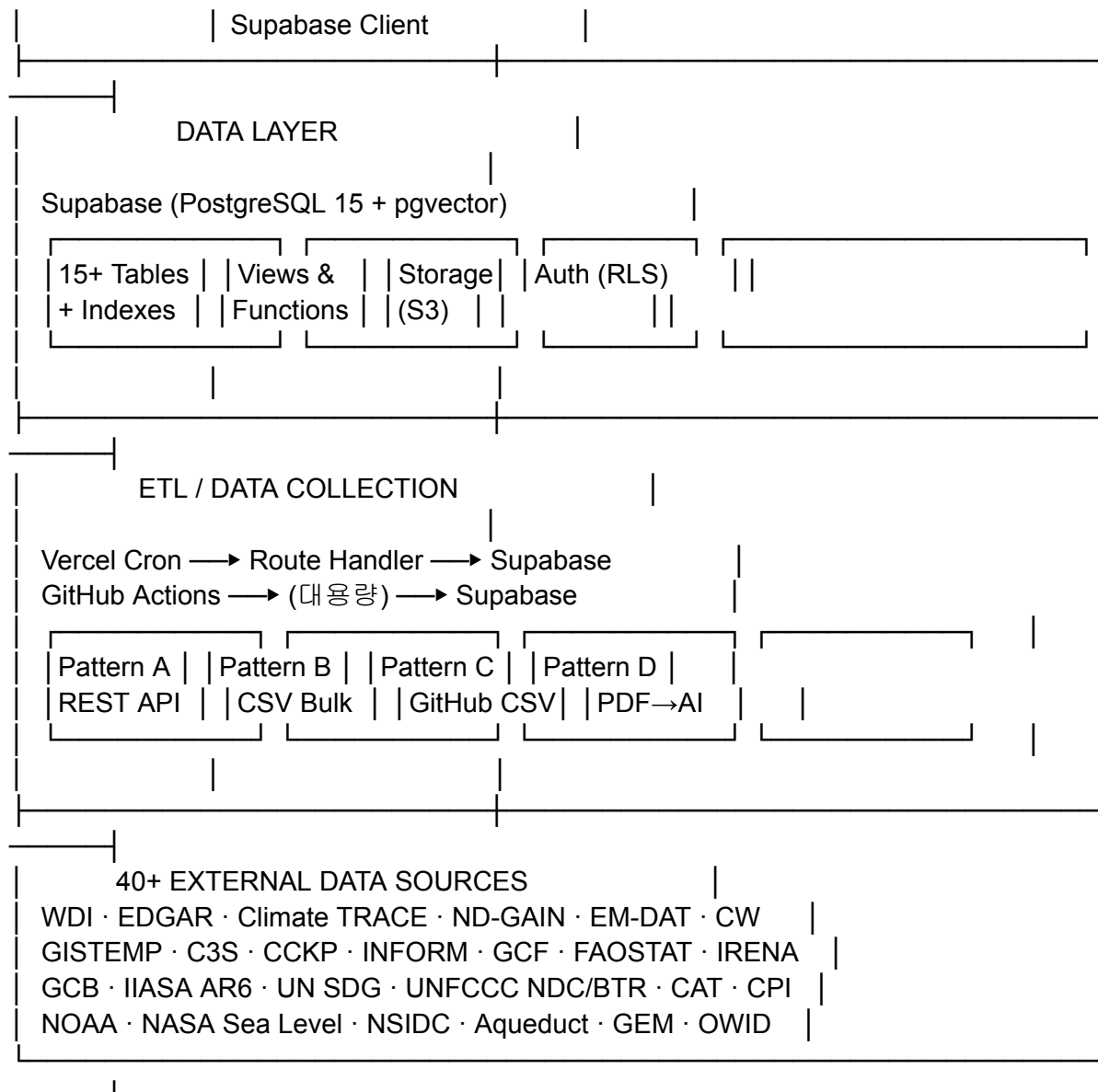
1.3 기술 스택

계층	기술	역할
프론트엔드	Next.js 15 (App Router, RSC, Server Actions)	ISR/SSR, 라우팅, API
UI	Tailwind CSS 4 + shadcn/ui	컴포넌트 시스템
차트	Recharts (표준) + D3.js v7 (커스텀)	시계열, 비교, 산점도
지도	Mapbox GL JS	Choropleth, 자산 위치
i18n	next-intl 4.x	6개 UN 공식 언어
PWA	Servist	오프라인, 설치 가능
DB	Supabase (PostgreSQL 15+)	관계형 저장, RLS
파일	Supabase Storage	원시 CSV/PDF 보관
인증	Supabase Auth	OAuth 2.0, Magic Link
AI	Claude API (claude-sonnet-4)	RAG 기반 자연어 질의

ETL	Vercel Cron + Route Handler + Supabase Edge Function	스케줄 수집
배포	Vercel	Edge Network, 서버리스
CI/CD	GitHub Actions	테스트, 빌드, 배포
모니터링	Vercel Analytics + Sentry	성능, 에러 추적

1.4 아키텍처 다이어그램





1.5 핵심 설계 원칙

P1 — 원천 보존: 수집 원시 값을 변환 없이 **DLO** 테이블에 저장. 단위 변환·GWP 환산·집계는 파생 뷰/함수에서만.

P2 — 복수 소스 비집계: 동일 지표에 여러 소스가 다른 값을 줄 때 절대 평균하지 않음. **source_id** 컬럼으로 구분하여 병렬 보존.

P3 — 역등성: 동일 소스·날짜로 ETL 재실행 시 결과 동일. **UPSERT + SHA-256** 체크섬.

P4 — 금지 변환: GWP 기준 명시 없는 **CO₂eq** 환산 금지, 소스 수준 세분성 제거 금지, 자동 보간 금지.

P5 — 감사 추적: 매 ETL 실행마다 `etl_logs` 레코드 생성 (`run_id`, 소스, 시간, 건수, SHA-256, 상태).

Part 2 — 데이터베이스 설계

2.1 ER 다이어그램

countries



data_sources



2.2 테이블 정의

countries

```
CREATE TABLE countries (  
  iso3          CHAR(3)    PRIMARY KEY,  
  iso2          CHAR(2)    UNIQUE NOT NULL,  
  name_en       TEXT      NOT NULL,  
  name_fr       TEXT,  
  name_es       TEXT,  
  name_ar       TEXT,  
  name_zh       TEXT,  
  name_ru       TEXT,  
  region_wb     TEXT,      -- World Bank region  
  subregion_un  TEXT,      -- UN subregion  
  income_group  TEXT,      -- WB income group  
  latitude      DECIMAL(9,6),  
  longitude     DECIMAL(9,6),  
  is_annex1     BOOLEAN DEFAULT FALSE,  
  is_non_annex1 BOOLEAN DEFAULT FALSE,  
  is_sids       BOOLEAN DEFAULT FALSE,  
  is_ldc        BOOLEAN DEFAULT FALSE,  
  is_lldc       BOOLEAN DEFAULT FALSE, -- Landlocked Developing  
  unfccc_group  TEXT,      -- 'G77','AOSIS','EU','AILAC','LMDCs', etc.  
  population_latest BIGINT,  
  gdp_latest_usd DECIMAL(18,2),  
  area_km2      DECIMAL(12,2),  
  updated_at    TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE INDEX idx_countries_region ON countries(region_wb);  
CREATE INDEX idx_countries_income ON countries(income_group);  
CREATE INDEX idx_countries_groups ON countries(is_annex1, is_sids, is_ldc);
```

data_sources

```
CREATE TABLE data_sources (  
  source_id     TEXT    PRIMARY KEY, -- 'DS-01'...'DS-40'  
  name          TEXT    NOT NULL,  
  name_short    TEXT,  
  organization  TEXT    NOT NULL,  
  url           TEXT,  
  api_endpoint  TEXT,  
  license       TEXT    NOT NULL,  
  license_url   TEXT,  
  can_expose_raw  BOOLEAN DEFAULT TRUE,  
  can_redistribute BOOLEAN DEFAULT TRUE,
```

```

collection_pattern CHAR(1) NOT NULL CHECK (collection_pattern IN ('A','B','C','D')),
frequency          TEXT,
gwp_basis          TEXT,
methodology_summary TEXT,
known_limitations  TEXT,
citation           TEXT,
data_start_year    SMALLINT,
data_end_year      SMALLINT,
country_coverage   SMALLINT, -- 국가 수
estimated_records  INTEGER,
created_at         TIMESTAMPTZ DEFAULT NOW()
);

```

emissions

```

CREATE TABLE emissions (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  country_iso3 CHAR(3) NOT NULL REFERENCES countries(iso3),
  source_id   TEXT NOT NULL REFERENCES data_sources(source_id),
  gas         TEXT NOT NULL,
  sector      TEXT,
  subsector   TEXT,
  year        SMALLINT NOT NULL,
  value       DECIMAL(18,6),
  unit        TEXT NOT NULL,
  gwp_basis   TEXT,
  is_preliminary BOOLEAN DEFAULT FALSE,
  retrieved_at TIMESTAMPTZ NOT NULL,
  file_sha256 TEXT,
  UNIQUE (source_id, country_iso3, gas, sector, subsector, year)
);

```

```

CREATE INDEX idx_em_country_year ON emissions(country_iso3, year);
CREATE INDEX idx_em_source ON emissions(source_id);
CREATE INDEX idx_em_gas ON emissions(gas);
CREATE INDEX idx_em_sector ON emissions(sector) WHERE sector IS NOT NULL;
CREATE INDEX idx_em_year ON emissions(year);
-- 복합 인덱스: 국가 프로필 쿼리 최적화
CREATE INDEX idx_em_profile ON emissions(country_iso3, gas, source_id, year)
  WHERE sector IS NULL;

```

climate_indicators

```

CREATE TABLE climate_indicators (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  country_iso3 CHAR(3) REFERENCES countries(iso3),
  source_id   TEXT NOT NULL REFERENCES data_sources(source_id),

```

```

indicator    TEXT      NOT NULL,
region       TEXT,
year         SMALLINT  NOT NULL,
month        SMALLINT,
value        DECIMAL(12,4),
unit         TEXT      NOT NULL,
baseline_period TEXT,
scenario      TEXT,
model        TEXT,
retrieved_at TIMESTAMPTZ NOT NULL,
UNIQUE (source_id, country_iso3, indicator, region, year, month, scenario, model)
);

```

```

CREATE INDEX idx_ci_country ON climate_indicators(country_iso3);
CREATE INDEX idx_ci_indicator ON climate_indicators(indicator);

```

vulnerability

```

CREATE TABLE vulnerability (
  id          BIGINT    GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  country_iso3 CHAR(3)  NOT NULL REFERENCES countries(iso3),
  source_id   TEXT      NOT NULL REFERENCES data_sources(source_id),
  indicator   TEXT      NOT NULL,
  component   TEXT,
  subcomponent TEXT,
  year        SMALLINT  NOT NULL,
  value       DECIMAL(12,6),
  unit        TEXT,
  retrieved_at TIMESTAMPTZ NOT NULL,
  UNIQUE (source_id, country_iso3, indicator, component, subcomponent, year)
);

```

```

CREATE INDEX idx_vuln_country ON vulnerability(country_iso3);

```

climate_finance

```

CREATE TABLE climate_finance (
  id          BIGINT    GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  country_iso3 CHAR(3)  REFERENCES countries(iso3),
  source_id   TEXT      NOT NULL REFERENCES data_sources(source_id),
  flow_type   TEXT,      -- 'mitigation','adaptation','cross-cutting','loss_and_damage'
  channel     TEXT,      -- 'multilateral','bilateral','private','domestic'
  instrument  TEXT,      -- 'grant','concessional_loan','equity','guarantee'
  provider    TEXT,
  fund_name   TEXT,
  year        SMALLINT  NOT NULL,
  amount_usd  DECIMAL(18,2),

```

```

currency_original TEXT,
amount_original DECIMAL(18,2),
retrieved_at TIMESTAMPTZ NOT NULL,
UNIQUE (source_id, country_iso3, flow_type, channel, instrument, provider, fund_name,
year)
);

```

```

CREATE INDEX idx_fin_country ON climate_finance(country_iso3);
CREATE INDEX idx_fin_type ON climate_finance(flow_type);

```

disasters

```

CREATE TABLE disasters (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  disaster_no TEXT UNIQUE NOT NULL,
  country_iso3 CHAR(3) NOT NULL REFERENCES countries(iso3),
  source_id TEXT NOT NULL REFERENCES data_sources(source_id),
  disaster_group TEXT, -- 'Natural','Technological'
  disaster_type TEXT NOT NULL,
  disaster_subtype TEXT,
  year SMALLINT NOT NULL,
  start_date DATE,
  end_date DATE,
  total_deaths INTEGER,
  total_injured INTEGER,
  total_affected BIGINT,
  total_homeless BIGINT,
  total_damage_usd DECIMAL(18,2),
  insured_damage_usd DECIMAL(18,2),
  is_climate_related BOOLEAN DEFAULT FALSE,
  retrieved_at TIMESTAMPTZ NOT NULL
);

```

```

CREATE INDEX idx_dis_country ON disasters(country_iso3);
CREATE INDEX idx_dis_type ON disasters(disaster_type);
CREATE INDEX idx_dis_climate ON disasters(is_climate_related) WHERE
is_climate_related;

```

ndc_targets (구조화된 NDC 목표)

```

CREATE TABLE ndc_targets (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  country_iso3 CHAR(3) NOT NULL REFERENCES countries(iso3),
  source_id TEXT NOT NULL REFERENCES data_sources(source_id),
  ndc_version TEXT NOT NULL,
  submission_date DATE,

```

```

target_type      TEXT,      --
'absolute','BAU_reduction','intensity','peaking','carbon_budget'
base_year        SMALLINT,
base_year_emissions DECIMAL(18,2),
target_year      SMALLINT,
target_value     DECIMAL(18,2),
reduction_pct    DECIMAL(6,2),
reduction_pct_conditional DECIMAL(6,2),
conditionality   TEXT,
sector_scope     TEXT,
gas_scope        TEXT,
lulucf_included  BOOLEAN,
international_markets BOOLEAN,
net_zero_target_year SMALLINT,
net_zero_legislated  BOOLEAN,
document_url      TEXT,
extraction_confidence TEXT CHECK (extraction_confidence IN ('high','medium','low')),
retrieved_at      TIMESTAMPTZ NOT NULL,
UNIQUE (source_id, country_iso3, ndc_version, target_year)
);

```

ndc_content (Climate Watch NDC Content — 150+ 지표)

```

CREATE TABLE ndc_content (
  id          BIGINT  GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  country_iso3 CHAR(3) NOT NULL REFERENCES countries(iso3),
  ndc_version  TEXT    NOT NULL,
  global_category TEXT  NOT NULL,
  overview_category TEXT,
  sector       TEXT,
  subsector    TEXT,
  indicator_id TEXT    NOT NULL,
  indicator_name TEXT   NOT NULL,
  value        TEXT,    -- free-text value from NDC
  retrieved_at TIMESTAMPTZ NOT NULL,
  UNIQUE (country_iso3, ndc_version, indicator_id, sector, subsector)
);

```

```

CREATE INDEX idx_ndc_content_country ON ndc_content(country_iso3);
CREATE INDEX idx_ndc_content_indicator ON ndc_content(indicator_id);

```

btr_status (BTR 제출 현황 추적)

```

CREATE TABLE btr_status (
  id          BIGINT  GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  country_iso3 CHAR(3) NOT NULL REFERENCES countries(iso3),
  btr_version  TEXT    NOT NULL DEFAULT 'BTR1',

```

```

submitted      BOOLEAN DEFAULT FALSE,
submission_date DATE,
-- 구성 요소별 제출 여부
has_nir         BOOLEAN, -- National Inventory Report
has_crt         BOOLEAN, -- Common Reporting Tables
has_ctf         BOOLEAN, -- Common Tabular Formats
has_ndc_tracking BOOLEAN, -- NDC progress tracking
has_adaptation  BOOLEAN, -- Adaptation chapter
has_finance_support BOOLEAN, -- Finance/support info
has_redd_plus   BOOLEAN, -- REDD+ annex
has_article6    BOOLEAN, -- Article 6 cooperation
-- 품질 지표
inventory_years TEXT,    -- e.g. '1990-2022'
gases_covered   TEXT,    -- 'CO2,CH4,N2O,HFCs,PFCs,SF6,NF3'
sectors_covered TEXT,    -- 'Energy,IPPU,Agriculture,LULUCF,Waste'
document_url    TEXT,
synthesis_notes TEXT,    -- 2025 synthesis report 기반 요약
retrieved_at    TIMESTAMPTZ NOT NULL,
UNIQUE (country_iso3, btr_version)
);

```

```

CREATE INDEX idx_btr_submitted ON btr_status(submitted);

```

nap_status (국가 적응 계획 현황)

```

CREATE TABLE nap_status (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  country_iso3 CHAR(3) NOT NULL REFERENCES countries(iso3),
  has_nap     BOOLEAN DEFAULT FALSE,
  nap_submission_date DATE,
  nap_document_url TEXT,
  -- 적응 초점 영역
  focus_agriculture BOOLEAN,
  focus_water        BOOLEAN,
  focus_health       BOOLEAN,
  focus_infrastructure BOOLEAN,
  focus_ecosystems   BOOLEAN,
  focus_coastal      BOOLEAN,
  focus_energy       BOOLEAN,
  focus_urban        BOOLEAN,
  -- 연계
  nap_ndc_aligned   BOOLEAN, -- NAP가 NDC 적응 목표와 정렬되었는가
  monitoring_framework BOOLEAN, -- M&E 프레임워크 존재
  retrieved_at      TIMESTAMPTZ NOT NULL,
  UNIQUE (country_iso3)
);

```

reduction_potential (Climate TRACE 감축 잠재량)

```
CREATE TABLE reduction_potential (  
  id          BIGINT    GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  country_iso3 CHAR(3)  NOT NULL REFERENCES countries(iso3),  
  source_id   TEXT     NOT NULL REFERENCES data_sources(source_id),  
  sector      TEXT     NOT NULL,  
  subsector   TEXT,  
  strategy    TEXT     NOT NULL, -- 감축 전략명  
  current_emissions DECIMAL(18,3),  
  reduction_potential DECIMAL(18,3),  
  unit        TEXT DEFAULT 'tonnes CO2eq/year',  
  year        SMALLINT,  
  asset_count INTEGER,          -- 해당 전략 적용 가능 자산 수  
  retrieved_at TIMESTAMPTZ NOT NULL,  
  UNIQUE (source_id, country_iso3, sector, subsector, strategy, year)  
);
```

```
CREATE INDEX idx_rp_country ON reduction_potential(country_iso3);  
CREATE INDEX idx_rp_sector ON reduction_potential(sector);
```

country_projections (기후 전망)

```
CREATE TABLE country_projections (  
  id          BIGINT    GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  country_iso3 CHAR(3)  NOT NULL REFERENCES countries(iso3),  
  source_id   TEXT     NOT NULL REFERENCES data_sources(source_id),  
  variable    TEXT     NOT NULL, -- 'tas','pr','tasmax','tasmin'  
  scenario    TEXT     NOT NULL, -- 'SSP1-2.6','SSP2-4.5','SSP3-7.0','SSP5-8.5'  
  model       TEXT,      -- 'ensemble' or specific CMIP6 model  
  period      TEXT     NOT NULL, -- '2020-2039','2040-2059','2060-2079','2080-2099'  
  aggregation TEXT     NOT NULL, -- 'annual','seasonal','monthly'  
  statistic   TEXT     NOT NULL, -- 'mean','median','p10','p90'  
  value       DECIMAL(10,4),  
  anomaly     DECIMAL(10,4),    -- baseline 대비 변화  
  unit        TEXT     NOT NULL,  
  baseline_period TEXT,  
  retrieved_at TIMESTAMPTZ NOT NULL,  
  UNIQUE (source_id, country_iso3, variable, scenario, model, period, aggregation, statistic)  
);
```

etl_logs, quarantine

```
CREATE TABLE etl_logs (  
  run_id      UUID      PRIMARY KEY DEFAULT gen_random_uuid(),  
  source_id   TEXT     NOT NULL REFERENCES data_sources(source_id),  
  started_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```

completed_at TIMESTAMPTZ,
status       TEXT       NOT NULL DEFAULT 'running'
              CHECK (status IN ('running','success','partial','failed')),
record_count INTEGER    DEFAULT 0,
inserted_count INTEGER   DEFAULT 0,
updated_count INTEGER    DEFAULT 0,
skipped_count INTEGER    DEFAULT 0,
quarantine_count INTEGER  DEFAULT 0,
file_sha256   TEXT,
file_size_bytes BIGINT,
api_url       TEXT,
error_message TEXT,
duration_ms   INTEGER,
triggered_by  TEXT       DEFAULT 'cron'
);

CREATE TABLE quarantine (
  id          BIGINT      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  run_id      UUID        REFERENCES etl_logs(run_id),
  source_id   TEXT        NOT NULL,
  raw_data    JSONB        NOT NULL,
  error_type  TEXT        NOT NULL,
  error_detail TEXT,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  resolved_at TIMESTAMPTZ,
  resolved_by TEXT
);

```

2.3 RLS 정책

-- 모든 데이터 테이블: 공개 읽기 (비영리 공개 플랫폼)

```

DO $$
DECLARE
  tbl TEXT;
BEGIN
  FOR tbl IN SELECT unnest(ARRAY[
    'countries','data_sources','emissions','climate_indicators',
    'vulnerability','climate_finance','disasters','ndc_targets',
    'ndc_content','btr_status','nap_status','reduction_potential',
    'country_projections'
  ])
  LOOP
    EXECUTE format('ALTER TABLE %I ENABLE ROW LEVEL SECURITY', tbl);
    EXECUTE format(
      'CREATE POLICY "public_read" ON %I FOR SELECT USING (true)', tbl
    );
  END LOOP;

```

```
END $$;
```

```
-- 관리자 전용 테이블
```

```
ALTER TABLE etl_logs ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE quarantine ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY "admin_read" ON etl_logs FOR SELECT
```

```
  USING (auth.jwt() ->> 'role' = 'admin');
```

```
CREATE POLICY "admin_read" ON quarantine FOR SELECT
```

```
  USING (auth.jwt() ->> 'role' = 'admin');
```

2.4 핵심 뷰 & 함수

국가 기후 대시보드 통합 함수

```
CREATE OR REPLACE FUNCTION fn_country_dashboard(p_iso3 CHAR(3))
```

```
RETURNS JSONB AS $$
```

```
SELECT jsonb_build_object(
```

```
  -- 기본 정보
```

```
  'country', (
```

```
    SELECT to_jsonb(c) FROM countries c WHERE iso3 = p_iso3
```

```
  ),
```

```
  -- 최신 배출 (모든 소스)
```

```
  'emissions_latest', (
```

```
    SELECT jsonb_agg(to_jsonb(sub))
```

```
    FROM (
```

```
      SELECT e.source_id, ds.name AS source_name, e.gas, e.year,
```

```
             e.value, e.unit, e.gwp_basis, e.is_preliminary
```

```
      FROM emissions e
```

```
      JOIN data_sources ds ON e.source_id = ds.source_id
```

```
      WHERE e.country_iso3 = p_iso3 AND e.sector IS NULL
```

```
      AND e.year = (
```

```
        SELECT MAX(year) FROM emissions
```

```
        WHERE country_iso3 = p_iso3 AND sector IS NULL
```

```
      )
```

```
      ORDER BY e.source_id, e.gas
```

```
    ) sub
```

```
  ),
```

```
  -- 배출 시계열 (총계, CO2, 주요 소스만)
```

```
  'emissions_timeseries', (
```

```
    SELECT jsonb_agg(to_jsonb(sub))
```

```
    FROM (
```

```
      SELECT source_id, gas, year, value, unit
```

```
      FROM emissions
```

```
      WHERE country_iso3 = p_iso3 AND sector IS NULL AND gas IN ('CO2','GHG_total')
```

```

        ORDER BY source_id, gas, year
    ) sub
),

-- NDC 목표
'ndc', (
    SELECT to_jsonb(sub)
    FROM (
        SELECT * FROM ndc_targets
        WHERE country_iso3 = p_iso3
        ORDER BY submission_date DESC LIMIT 1
    ) sub
),

-- NDC 이행 격차 (최신 배출 vs 목표)
'ndc_gap', (
    SELECT jsonb_build_object(
        'latest_emission_year', e.year,
        'latest_emission_value', e.value,
        'target_year', n.target_year,
        'target_value', n.target_value,
        'base_year', n.base_year,
        'base_year_emissions', n.base_year_emissions,
        'reduction_required_pct', n.reduction_pct,
        'linear_projected_value',
        CASE WHEN n.target_year > e.year AND e.value IS NOT NULL
            THEN e.value + (e.value - LAG(e.value, 5) OVER (ORDER BY e.year))
                / 5.0 * (n.target_year - e.year)
            ELSE NULL
        END
    )
    FROM emissions e
    CROSS JOIN LATERAL (
        SELECT * FROM ndc_targets
        WHERE country_iso3 = p_iso3
        ORDER BY submission_date DESC LIMIT 1
    ) n
    WHERE e.country_iso3 = p_iso3 AND e.sector IS NULL
        AND e.gas = 'GHG_total' AND e.source_id = 'DS-05'
    ORDER BY e.year DESC LIMIT 1
),

-- BTR 현황
'btr', (
    SELECT to_jsonb(b) FROM btr_status b
    WHERE country_iso3 = p_iso3 AND btr_version = 'BTR1'
),

```

```

-- NAP 현황
'nap', (
  SELECT to_jsonb(n) FROM nap_status n
  WHERE country_iso3 = p_iso3
),

-- 취약성 (최신)
'vulnerability', (
  SELECT jsonb_agg(to_jsonb(sub))
  FROM (
    SELECT source_id, indicator, component, year, value
    FROM vulnerability
    WHERE country_iso3 = p_iso3
    AND indicator IN ('ndgain_index','ndgain_vulnerability','ndgain_readiness','inform_risk')
    AND year = (SELECT MAX(year) FROM vulnerability WHERE country_iso3 = p_iso3)
    ORDER BY indicator
  ) sub
),

-- 기후 자원 (최근 5년 합계)
'finance_5yr', (
  SELECT jsonb_build_object(
    'total_usd', SUM(amount_usd),
    'mitigation_usd', SUM(CASE WHEN flow_type = 'mitigation' THEN amount_usd ELSE 0
END),
    'adaptation_usd', SUM(CASE WHEN flow_type = 'adaptation' THEN amount_usd ELSE
0 END),
    'years', MIN(year) || '-' || MAX(year)
  )
  FROM climate_finance
  WHERE country_iso3 = p_iso3
  AND year >= EXTRACT(YEAR FROM NOW()) - 5
),

-- 기후 재해 (최근 10년)
'disasters_10yr', (
  SELECT jsonb_build_object(
    'count', COUNT(*),
    'total_deaths', SUM(total_deaths),
    'total_affected', SUM(total_affected),
    'total_damage_usd', SUM(total_damage_usd)
  )
  FROM disasters
  WHERE country_iso3 = p_iso3 AND is_climate_related
  AND year >= EXTRACT(YEAR FROM NOW()) - 10
),

-- 감축 잠재량 상위 5개

```

```

'top_reduction_strategies', (
  SELECT jsonb_agg(to_jsonb(sub))
  FROM (
    SELECT sector, strategy, reduction_potential, unit, asset_count
    FROM reduction_potential
    WHERE country_iso3 = p_iso3
    ORDER BY reduction_potential DESC LIMIT 5
  ) sub
),
-- 기후 전망 (기온, SSP2-4.5)
'temp_projection', (
  SELECT jsonb_agg(to_jsonb(sub))
  FROM (
    SELECT period, value, anomaly, scenario
    FROM country_projections
    WHERE country_iso3 = p_iso3
      AND variable = 'tas' AND scenario IN ('SSP2-4.5', 'SSP5-8.5')
      AND model = 'ensemble' AND aggregation = 'annual' AND statistic = 'median'
    ORDER BY scenario, period
  ) sub
)
);
$$ LANGUAGE SQL STABLE;

```

NDC 이행 격차 계산 함수

```

CREATE OR REPLACE FUNCTION fn_ndc_gap_analysis(p_iso3 CHAR(3))
RETURNS JSONB AS $$
WITH latest_ndc AS (
  SELECT * FROM ndc_targets
  WHERE country_iso3 = p_iso3
  ORDER BY submission_date DESC LIMIT 1
),
emission_trend AS (
  SELECT year, value,
    -- 최근 5년 평균 변화율
    AVG(value - LAG(value) OVER (ORDER BY year))
      OVER (ORDER BY year ROWS BETWEEN 4 PRECEDING AND CURRENT ROW) AS
    avg_annual_change
  FROM emissions
  WHERE country_iso3 = p_iso3 AND source_id = 'DS-05'
    AND gas = 'GHG_total' AND sector IS NULL
  ORDER BY year
),
latest_point AS (
  SELECT * FROM emission_trend ORDER BY year DESC LIMIT 1

```

```

),
projected AS (
  SELECT
    lp.year AS last_data_year,
    lp.value AS last_value,
    lp.avg_annual_change,
    n.target_year,
    n.base_year_emissions,
    n.reduction_pct,
    -- 목표 배출량
    n.base_year_emissions * (1 - n.reduction_pct / 100.0) AS target_emissions,
    -- 선형 외삽 배출량
    lp.value + lp.avg_annual_change * (n.target_year - lp.year) AS projected_emissions,
    -- 격차
    (lp.value + lp.avg_annual_change * (n.target_year - lp.year))
      - (n.base_year_emissions * (1 - n.reduction_pct / 100.0)) AS gap_value
  FROM latest_point lp, latest_ndc n
)
SELECT to_jsonb(p) FROM projected p;
$$ LANGUAGE SQL STABLE;

```

적응-취약성-재원 삼각 분석 뷰

```

CREATE OR REPLACE VIEW v_adaptation_triangle AS
SELECT
  c.iso3,
  c.name_en,
  c.region_wb,
  c.income_group,
  c.is_sids,
  c.is_ldc,
  -- 취약성 (ND-GAIN, 0-1, 높을수록 취약)
  vuln.ndgain_vulnerability,
  vuln.ndgain_readiness,
  vuln.ndgain_index,
  -- 위험 (INFORM, 0-10, 높을수록 위험)
  inf.inform_risk,
  -- 재원 (최근 5년 합계)
  fin.total_finance_usd,
  fin.finance_per_capita,
  -- 재해 (최근 10년)
  dis.disaster_count,
  dis.total_deaths,
  dis.total_affected,
  -- 격차 지표: 취약성 높지만 재원 적은 국가 식별
CASE
  WHEN vuln.ndgain_vulnerability > 0.5 AND fin.finance_per_capita < 10

```

```

    THEN TRUE ELSE FALSE
  END AS finance_gap_flag
FROM countries c
LEFT JOIN LATERAL (
  SELECT
    MAX(CASE WHEN indicator = 'ndgain_vulnerability' THEN value END) AS
ndgain_vulnerability,
    MAX(CASE WHEN indicator = 'ndgain_readiness' THEN value END) AS
ndgain_readiness,
    MAX(CASE WHEN indicator = 'ndgain_index' THEN value END) AS ndgain_index
  FROM vulnerability v
  WHERE v.country_iso3 = c.iso3
    AND v.year = (SELECT MAX(year) FROM vulnerability WHERE country_iso3 = c.iso3)
) vuln ON TRUE
LEFT JOIN LATERAL (
  SELECT MAX(value) AS inform_risk
  FROM vulnerability
  WHERE country_iso3 = c.iso3 AND indicator = 'inform_risk'
    AND year = (SELECT MAX(year) FROM vulnerability WHERE country_iso3 = c.iso3 AND
indicator = 'inform_risk')
) inf ON TRUE
LEFT JOIN LATERAL (
  SELECT
    SUM(amount_usd) AS total_finance_usd,
    CASE WHEN c.population_latest > 0
      THEN SUM(amount_usd) / c.population_latest
      ELSE NULL
    END AS finance_per_capita
  FROM climate_finance
  WHERE country_iso3 = c.iso3 AND year >= EXTRACT(YEAR FROM NOW()) - 5
) fin ON TRUE
LEFT JOIN LATERAL (
  SELECT
    COUNT(*) AS disaster_count,
    SUM(total_deaths) AS total_deaths,
    SUM(total_affected) AS total_affected
  FROM disasters
  WHERE country_iso3 = c.iso3 AND is_climate_related
    AND year >= EXTRACT(YEAR FROM NOW()) - 10
) dis ON TRUE;

```

Part 3 — 데이터 수집 파이프라인 (ETL)

3.1 공통 프레임워크

```
// lib/etl/base.ts
```

```
import { createClient } from '@supabase/supabase-js';
import { createHash } from 'crypto';
```

```
const supabaseAdmin = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!
);
```

```
export { supabaseAdmin };
```

```
export interface EtlResult {
  runId: string;
  sourceId: string;
  status: 'success' | 'partial' | 'failed';
  recordCount: number;
  insertedCount: number;
  updatedCount: number;
  skippedCount: number;
  quarantineCount: number;
  durationMs: number;
  error?: string;
}
```

```
export interface EtlModule {
  sourceId: string;
  run(): Promise<Omit<EtlResult, 'runId' | 'durationMs'>>;
}
```

```
export async function executeEtl(mod: EtlModule): Promise<EtlResult> {
  const t0 = Date.now();
  const { data: log } = await supabaseAdmin
    .from('etl_logs')
    .insert({ source_id: mod.sourceId, status: 'running' })
    .select('run_id')
    .single();
  const runId = log!.run_id;

  try {
    const result = await mod.run();
    const ms = Date.now() - t0;
    await supabaseAdmin.from('etl_logs').update({
      status: result.status,
      completed_at: new Date().toISOString(),
      record_count: result.recordCount,
    });
  } catch (error) {
    // Handle error
  }
}
```

```

    inserted_count: result.insertedCount,
    updated_count: result.updatedCount,
    skipped_count: result.skippedCount,
    quarantine_count: result.quarantineCount,
    duration_ms: ms,
  }).eq('run_id', runId);
  return { ...result, runId, durationMs: ms };
} catch (err) {
  const ms = Date.now() - t0;
  const msg = err instanceof Error ? err.message : String(err);
  await supabaseAdmin.from('etl_logs').update({
    status: 'failed', completed_at: new Date().toISOString(),
    duration_ms: ms, error_message: msg.slice(0, 2000),
  }).eq('run_id', runId);
  return { runId, sourceId: mod.sourceId, status: 'failed',
    recordCount: 0, insertedCount: 0, updatedCount: 0,
    skippedCount: 0, quarantineCount: 0, durationMs: ms, error: msg };
}
}

export async function batchUpsert(
  table: string, rows: Record<string, unknown>[], conflictCols: string[],
  batchSize = 500
): Promise<{ total: number }> {
  let total = 0;
  for (let i = 0; i < rows.length; i += batchSize) {
    const chunk = rows.slice(i, i + batchSize);
    const { data, error } = await supabaseAdmin
      .from(table)
      .upsert(chunk, { onConflict: conflictCols.join(',') })
      .select('id');
    if (error) throw new Error(`Upsert ${table}: ${error.message}`);
    total += data?.length ?? 0;
  }
  return { total };
}

export function sha256(text: string): string {
  return createHash('sha256').update(text).digest('hex');
}

export function sleep(ms: number) {
  return new Promise(r => setTimeout(r, ms));
}

```

3.2 DS-01 World Bank WDI

```

// lib/etl/wdi.ts
import { EtlModule, batchUpsert, sleep } from './base';
import { z } from 'zod';

const INDICATORS: Record<string, { gas: string; unit: string; table: string }> = {
  'EN.ATM.CO2E.KT': { gas: 'CO2', unit: 'kt', table: 'emissions' },
  'EN.ATM.CO2E.PC': { gas: 'CO2_pc', unit: 'metric tons per capita', table: 'emissions' },
  'EN.ATM.GHGT.KT.CE': { gas: 'GHG_total', unit: 'kt CO2eq', table: 'emissions' },
  'EG.USE.PCAP.KG.OE': { gas: 'energy_use', unit: 'kg oil eq/capita', table:
'climate_indicators' },
  'EG.FEC.RNEW.ZS': { gas: 'renewable_pct', unit: '%', table: 'climate_indicators' },
  'NY.GDP.MKTP.CD': { gas: 'gdp', unit: 'current USD', table: 'meta' },
  'SP.POP.TOTL': { gas: 'population', unit: 'persons', table: 'meta' },
  'AG.LND.FRST.ZS': { gas: 'forest_pct', unit: '% of land area', table: 'climate_indicators' },
};

const BASE = 'https://api.worldbank.org/v2/country/all/indicator';

export class WdiEtl implements EtlModule {
  sourceId = 'DS-01';

  async run() {
    let inserted = 0, skipped = 0, quarantined = 0, total = 0;
    const emissionRows: Record<string, unknown>[] = [];
    const climateRows: Record<string, unknown>[] = [];

    for (const [code, meta] of Object.entries(INDICATORS)) {
      let page = 1, pages = 1;
      while (page <= pages) {
        const url =
`${BASE}/${code}?format=json&per_page=500&date=1990:2024&page=${page}`;
        const res = await fetch(url);
        if (res.status === 429) { await sleep(3000); continue; }
        if (!res.ok) throw new Error(`WDI ${res.status} for ${code}`);

        const json = await res.json();
        if (!Array.isArray(json) || json.length < 2) { page++; continue; }
        pages = json[0].pages;

        for (const rec of json[1]) {
          total++;
          if (!rec.countryiso3code || rec.countryiso3code.length !== 3) { quarantined++; continue; }

          if (rec.value === null) { skipped++; continue; }

          if (meta.table === 'emissions') {
            emissionRows.push({
              country_iso3: rec.countryiso3code,

```

```

    source_id: 'DS-01',
    gas: meta.gas,
    sector: null,
    subsector: null,
    year: parseInt(rec.date),
    value: rec.value,
    unit: meta.unit,
    gwp_basis: null,
    is_preliminary: false,
    retrieved_at: new Date().toISOString(),
  });
} else if (meta.table === 'climate_indicators') {
  climateRows.push({
    country_iso3: rec.countryiso3code,
    source_id: 'DS-01',
    indicator: meta.gas,
    region: null,
    year: parseInt(rec.date),
    month: null,
    value: rec.value,
    unit: meta.unit,
    baseline_period: null,
    scenario: null,
    model: null,
    retrieved_at: new Date().toISOString(),
  });
} else if (meta.table === 'meta') {
  // population, GDP → countries 테이블 업데이트 (별도 로직)
}
}
page++;
await sleep(100);
}
}

```

```

if (emissionRows.length > 0) {
  const r = await batchUpsert('emissions', emissionRows,
    ['source_id', 'country_iso3', 'gas', 'sector', 'subsector', 'year']);
  inserted += r.total;
}
if (climateRows.length > 0) {
  const r = await batchUpsert('climate_indicators', climateRows,
    ['source_id', 'country_iso3', 'indicator', 'region', 'year', 'month', 'scenario', 'model']);
  inserted += r.total;
}

```

```

return { sourceId: this.sourceId, status: quarantined > 0 ? 'partial' as const : 'success' as const,

```

```

    recordCount: total, insertedCount: inserted, updatedCount: 0,
    skippedCount: skipped, quarantineCount: quarantined };
  }
}

```

3.3 DS-06 Climate Watch NDC Content

이것이 **Visual Climate** 차별화의 핵심 데이터이다. Climate Watch API의 NDC Content 엔드포인트는 국가별 150+ 구조화된 NDC 지표를 제공한다.

```

// lib/etl/climate-watch-ndc.ts
import { EtlModule, supabaseAdmin, batchUpsert, sleep } from './base';

const CW_NDC_API = 'https://www.climatewatchdata.org/api/v1/data/ndc_content';

export class ClimateWatchNdcEtl implements EtlModule {
  sourceId = 'DS-06-NDC';

  async run() {
    let inserted = 0, total = 0;

    // 국가 목록 가져오기
    const { data: countries } = await supabaseAdmin
      .from('countries').select('iso3');

    for (const country of countries ?? []) {
      const url = `${CW_NDC_API}?countries[]=${country.iso3}`;
      const res = await fetch(url);
      if (!res.ok) continue;

      const json = await res.json();
      const rows = (json.data ?? []).map((d: any) => ({
        country_iso3: d.iso_code3,
        ndc_version: inferVersion(d),
        global_category: d.global_category,
        overview_category: d.overview_category,
        sector: d.sector,
        subsector: d.subsector,
        indicator_id: d.indicator_id,
        indicator_name: d.indicator_name,
        value: d.value,
        retrieved_at: new Date().toISOString(),
      }));

      total += rows.length;
      if (rows.length > 0) {

```

```

    const r = await batchUpsert('ndc_content', rows,
      ['country_iso3', 'ndc_version', 'indicator_id', 'sector', 'subsector']);
    inserted += r.total;
  }

  await sleep(200); // Rate limiting 예의
}

return { sourceId: this.sourceId, status: 'success' as const,
  recordCount: total, insertedCount: inserted, updatedCount: 0,
  skippedCount: 0, quarantineCount: 0 };
}
}

function inferVersion(record: any): string {
  // Climate Watch는 NDC 버전을 source 필드로 구분
  const src = record.source || "";
  if (src.includes('3rd') || src.includes('NDC 3.0')) return 'NDC3';
  if (src.includes('Updated') || src.includes('2nd')) return 'NDC2';
  return 'NDC1';
}

```

3.4 DS-BTR UNFCCC BTR 현황 수집

BTR 제출 현황은 UNFCCC 웹사이트에서 스크래핑 또는 synthesis report 기반으로 구조화한다.

```

// lib/etl/btr-status.ts
import { EtlModule, supabaseAdmin, batchUpsert } from './base';

// BTR 제출 현황은 UNFCCC synthesis report (2025)에서 추출한 정적 데이터셋 기반
// 실시간 API가 없으므로 CSV 기반으로 관리
// 파일: data/btr_status_2025.csv

import { parse } from 'csv-parse/sync';
import { readFileSync } from 'fs';
import path from 'path';

export class BtrStatusEtl implements EtlModule {
  sourceId = 'DS-BTR';

  async run() {
    const csvPath = path.join(process.cwd(), 'data', 'btr_status_2025.csv');
    const csvText = readFileSync(csvPath, 'utf-8');
    const records = parse(csvText, { columns: true, trim: true });
  }
}

```

```

const rows = records.map((r: any) => ({
  country_iso3: r.iso3,
  btr_version: 'BTR1',
  submitted: r.submitted === 'TRUE',
  submission_date: r.submission_date || null,
  has_nir: r.has_nir === 'TRUE',
  has_crt: r.has_crt === 'TRUE',
  has_ctf: r.has_ctf === 'TRUE',
  has_ndc_tracking: r.has_ndc_tracking === 'TRUE',
  has_adaptation: r.has_adaptation === 'TRUE',
  has_finance_support: r.has_finance_support === 'TRUE',
  has_redd_plus: r.has_redd_plus === 'TRUE',
  has_article6: r.has_article6 === 'TRUE',
  inventory_years: r.inventory_years,
  gases_covered: r.gases_covered,
  sectors_covered: r.sectors_covered,
  document_url: r.document_url,
  retrieved_at: new Date().toISOString(),
}));

const { total } = await batchUpsert('btr_status', rows,
  ['country_iso3', 'btr_version']);

return { sourceId: this.sourceId, status: 'success' as const,
  recordCount: rows.length, insertedCount: total, updatedCount: 0,
  skippedCount: 0, quarantineCount: 0 };
}
}

```

3.5 DS-02 Climate TRACE (감축 잠재량 포함)

```

// lib/etl/climate-trace.ts
import { EtlModule, batchUpsert, sha256 } from './base';

const CT_DOWNLOAD = 'https://downloads.climate TRACE.org/latest';
const CT_ERS = 'https://downloads.climate TRACE.org/latest/ers/ers_plan_global.zip';

export class ClimateTraceEtl implements EtlModule {
  sourceId = 'DS-02';

  async run() {
    let inserted = 0, total = 0;

    // 1. 국가별 배출 데이터 (CSV 다운로드)
    // Climate TRACE는 국가별 CSV를 제공
    // 여기서는 country-level summary를 처리

```

```

const res = await fetch(`${CT_DOWNLOAD}/country_emissions.csv`);
if (!res.ok) throw new Error(`CT download failed: ${res.status}`);
const csvText = await res.text();

// CSV 파싱 및 emissions 테이블에 upsert
// (EDGAR ETL과 유사한 로직 — 생략하지 않고 핵심만)
const { parse } = await import('csv-parse/sync');
const records = parse(csvText, { columns: true, trim: true });

const emissionRows = records
  .filter((r: any) => r.iso3_country && r.iso3_country.length === 3)
  .map((r: any) => ({
    country_iso3: r.iso3_country,
    source_id: 'DS-02',
    gas: r.gas || 'CO2eq',
    sector: r.sector,
    subsector: r.subsector,
    year: parseInt(r.year),
    value: parseFloat(r.emissions_quantity),
    unit: 'tonnes CO2eq',
    gwp_basis: 'AR5',
    is_preliminary: false,
    retrieved_at: new Date().toISOString(),
  }));

total += emissionRows.length;
const r1 = await batchUpsert('emissions', emissionRows,
  ['source_id', 'country_iso3', 'gas', 'sector', 'subsector', 'year']);
inserted += r1.total;

// 2. 감축 잠재량 데이터 (Top Emission Reduction Strategies)
const ersRes = await fetch(CT_ERS);
if (ersRes.ok) {
  // ZIP 내 CSV 처리 (실제 구현에서는 unzipper 사용)
  // reduction_potential 테이블에 upsert
  // 핵심 필드: country_iso3, sector, strategy, current_emissions, reduction_potential
}

return { sourceId: this.sourceId, status: 'success' as const,
  recordCount: total, insertedCount: inserted, updatedCount: 0,
  skippedCount: 0, quarantineCount: 0 };
}
}

```

3.6 나머지 ETL 모듈 요약

모든 ETL 모듈은 **EtlModule** 인터페이스를 구현하며, 동일한 패턴을 따른다.

모듈	소스	대상 테이블	특이사항
edgar.ts	DS-05 EDGAR CSV	emissions	wide→long 변환, 가스별 분할 실행
ndgain.ts	DS-04 ND-GAIN ZIP	vulnerability	6개 CSV(gain, vulnerability, readiness + 하위) 파싱
emdat.ts	DS-03 EM-DAT CSV	disasters	is_climate_related 자동 태깅 (Flood, Storm, Drought, Wildfire 등)
gistemp.ts	DS-09 GISTEMP CSV	climate_indicators	월별+연간 파싱, 기준기간 1951-1980 메타
cckp.ts	DS-18 CCKP API	country_projections	국가별 4 SSP × 4 period × 5 variable = 80 API 호출/국가
inform.ts	DS-14 INFORM API	vulnerability	JSON API, 50+ 하위 지표
gcf.ts	DS-15 GCF API	climate_finance	API Key 필요, 승인 프로젝트 기반
faostat.ts	DS-11 FAOSTAT bulk	emissions	농업 배출 전용, domain별 별도 다운로드
irena.ts	DS-12 IRENA CSV	climate_indicators	재생에너지 용량/발전량
gcb.ts	DS-13 GCB Excel	emissions	국가별 화석연료 CO ₂ + 글로벌 탄소 수지
cat.ts	DS-27 CAT	ndc_targets	국가 등급(Critically insufficient ~ 1.5°C compatible) 포함
cpi.ts	DS-23 CPI CSV	climate_finance	기후 자원 흐름 (공공/민간, 완화/적응)
unsdg.ts	DS-17 UN SDG API	climate_indicators	SDG 13 하위 지표
nap.ts	DS-NAP	nap_status	NAPcentral.org 스크래핑 + 수동 CSV
owid.ts	DS-35 OWID GitHub	emissions, climate_indicators	통합 데이터셋, 교차검증용

3.7 ETL Route Handler & Cron 설정

```
// app/api/v1/etl/[source]/route.ts
```

```
import { NextRequest, NextResponse } from 'next/server';
import { executeEtl } from '@lib/etl/base';
import { WdiEtl } from '@lib/etl/wdi';
import { EdgarEtl } from '@lib/etl/edgar';
import { ClimateTraceEtl } from '@lib/etl/climate-trace';
import { ClimateWatchNdcEtl } from '@lib/etl/climate-watch-ndc';
import { BtrStatusEtl } from '@lib/etl/btr-status';
// ... 전체 모듈 import
```

```
const REGISTRY: Record<string, () => any> = {
  'DS-01': () => new WdiEtl(),
  'DS-02': () => new ClimateTraceEtl(),
  'DS-05': () => new EdgarEtl(),
  'DS-06-NDC': () => new ClimateWatchNdcEtl(),
  'DS-BTR': () => new BtrStatusEtl(),
  // ... 전체 모듈
};
```

```
export async function GET(
  req: NextRequest,
  { params }: { params: Promise<{ source: string }> }
) {
  // 인증
  const auth = req.headers.get('authorization');
  if (auth !== `Bearer ${process.env.CRON_SECRET}`) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }

  const { source } = await params;
  const factory = REGISTRY[source];
  if (!factory) return NextResponse.json({ error: 'Unknown source' }, { status: 404 });

  const result = await executeEtl(factory());
  return NextResponse.json(result, { status: result.status === 'failed' ? 500 : 200 });
}
```

```
// vercel.json — Cron 스케줄
```

```
{
  "crons": [
    { "path": "/api/v1/etl/DS-09", "schedule": "0 6 10 * *" },
    { "path": "/api/v1/etl/DS-01", "schedule": "0 6 1 1,7 * *" },
    { "path": "/api/v1/etl/DS-02", "schedule": "0 6 1 1,4,7,10 * *" },
    { "path": "/api/v1/etl/DS-05", "schedule": "0 6 15 3 * *" },
    { "path": "/api/v1/etl/DS-06-NDC", "schedule": "0 6 15 3 * *" },
  ]
}
```

```
{ "path": "/api/v1/etl/DS-BTR", "schedule": "0 6 1 1,7 *" },
{ "path": "/api/v1/etl/DS-04", "schedule": "0 6 15 3 *" },
{ "path": "/api/v1/etl/DS-03", "schedule": "0 6 1 1,4,7,10 *" },
{ "path": "/api/v1/etl/DS-14", "schedule": "0 6 15 3 *" },
{ "path": "/api/v1/etl/DS-15", "schedule": "0 6 1 1,4,7,10 *" },
{ "path": "/api/v1/etl/DS-18", "schedule": "0 6 15 3 *" },
{ "path": "/api/v1/etl/DS-11", "schedule": "0 6 15 3 *" },
{ "path": "/api/v1/etl/DS-12", "schedule": "0 6 15 3 *" },
{ "path": "/api/v1/etl/DS-13", "schedule": "0 6 15 3 *" },
{ "path": "/api/v1/etl/DS-35", "schedule": "0 6 15 3 *" }
]
}
```

Part 4 — 백엔드 API

4.1 API 설계 규칙

모든 엔드포인트 경로는 `/api/v1/` prefix를 사용한다. 응답은 항상 `{ data, meta }` 형태이며, `meta`에 소스 정보, 총 레코드 수, 타임스탬프를 포함한다. 에러 응답은 `{ error, details? }` 형태이다. 입력 검증은 Zod 스키마로 수행한다.

4.2 핵심 엔드포인트 명세

GET /api/v1/countries

Query Params:

region - World Bank region 필터
income - Income group 필터
group - 'annex1','sids','ldc','ldc'
q - 이름 검색 (ILIKE)

Response:

```
{
  data: Country[],
  meta: { count: number, timestamp: string }
}
```

GET /api/v1/emissions

Query Params:

country - ISO3 (필수)
gas - 'CO2','CH4','N2O','GHG_total','F-gases' (기본: 'CO2')

sector - IPCC 섹터 (없으면 총계)
source - source_id (없으면 모든 소스)
year_min - 시작 연도 (기본: 1990)
year_max - 끝 연도 (기본: 2024)

Response:

```
{
  data: {
    [source_id]: {
      source_name: string,
      organization: string,
      license: string,
      unit: string,
      gwp_basis: string | null,
      timeseries: { year: number, value: number, is_preliminary: boolean }[]
    }
  },
  meta: { country, gas, year_range, total_records, sources_count }
}
```

GET /api/v1/ndc

Query Params:

country - ISO3 (필수)

Response:

```
{
  data: {
    targets: NdcTarget[], // ndc_targets 테이블
    content: NdcContent[], // ndc_content 테이블 (150+ 지표)
    cat_rating: string | null, // Climate Action Tracker 등급
  },
  meta: { ... }
}
```

GET /api/v1/btr

Query Params:

country - ISO3 (선택, 없으면 전체)

Response:

```
{
  data: BtrStatus[],
  meta: {
    total_submitted: number,
    total_parties: number,
    submission_rate: string, -- e.g. "101/194 (52%)"
  }
}
```

```
}  
}
```

GET /api/v1/gap-analysis ★ (차별화 엔드포인트)

Query Params:

country - ISO3 (필수)
source - 배출 소스 (기본: 'DS-05')

Response:

```
{  
  data: {  
    country: string,  
    latest_emissions: { year, value, source_id },  
    ndc_target: { base_year, target_year, reduction_pct, target_emissions },  
    trend: {  
      avg_annual_change: number,  
      projected_target_year_value: number,  
      gap: number, // 양수=초과, 음수=달성  
      on_track: boolean,  
    },  
    historical_timeseries: { year, value }[],  
    projected_timeseries: { year, value }[], // 선형 외삽  
    target_line: { year, value }[], // base→target 직선  
    ar6_pathways: { // IIASA AR6 호환 경로  
      '1.5C': { year, value }[],  
      '2C': { year, value }[],  
      'current_policies': { year, value }[],  
    },  
  },  
  meta: { methodology: 'linear_extrapolation_5yr_avg', ... }  
}
```

GET /api/v1/adaptation-triangle ★ (차별화 엔드포인트)

Query Params:

region - 필터 (선택)
income - 필터 (선택)

Response:

```
{  
  data: AdaptationTrianglePoint[],  
  // 각 포인트: { iso3, name, vulnerability, inform_risk,  
  //           finance_per_capita, disaster_count, finance_gap_flag }  
  meta: { ... }  
}
```

GET /api/v1/reduction-potential ★

Query Params:

country - ISO3 (필수)

Response:

```
{
  data: {
    total_potential: number,
    unit: string,
    strategies: ReductionStrategy[],
    // 각 전략: { sector, strategy, current, potential, asset_count }
  },
  meta: { source: 'Climate TRACE ERS', ... }
}
```

GET /api/v1/country-dashboard ★ (통합 엔드포인트)

하나의 호출로 국가 기후 대시보드에 필요한 모든 데이터를 반환한다.

```
// app/api/v1/country-dashboard/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { createClient } from '@lib/supabase/server';

export async function GET(req: NextRequest) {
  const iso3 = req.nextUrl.searchParams.get('country');
  if (!iso3 || iso3.length !== 3) {
    return NextResponse.json({ error: 'country (ISO3) required' }, { status: 400 });
  }

  const supabase = await createClient();
  const { data, error } = await supabase.rpc('fn_country_dashboard', { p_iso3: iso3 });

  if (error) return NextResponse.json({ error: error.message }, { status: 500 });
  return NextResponse.json({ data, meta: { timestamp: new Date().toISOString() } });
}
```

4.3 API 응답 예시 — KOR Gap Analysis

```
{
  "data": {
    "country": "KOR",
    "latest_emissions": {
      "year": 2022,
      "value": 654200,
      "unit": "kt CO2eq",
    }
  }
}
```

```

    "source_id": "DS-05",
    "source_name": "EDGAR v2024"
  },
  "ndc_target": {
    "base_year": 2018,
    "base_year_emissions": 727600,
    "target_year": 2030,
    "reduction_pct": 40,
    "target_emissions": 436560,
    "ndc_version": "Updated NDC",
    "net_zero_target_year": 2050
  },
  "trend": {
    "avg_annual_change_kt": -9175,
    "projected_2030_value": 580800,
    "gap_kt": 144240,
    "on_track": false,
    "required_annual_reduction": -27205,
    "current_annual_reduction": -9175,
    "ambition_gap_factor": 2.96
  },
  "historical_timeseries": [
    { "year": 1990, "value": 292220 },
    { "year": 2000, "value": 502690 },
    { "year": 2010, "value": 656420 },
    { "year": 2018, "value": 727600 },
    { "year": 2022, "value": 654200 }
  ],
  "projected_timeseries": [
    { "year": 2022, "value": 654200 },
    { "year": 2025, "value": 626675 },
    { "year": 2030, "value": 580800 }
  ],
  "target_line": [
    { "year": 2018, "value": 727600 },
    { "year": 2030, "value": 436560 }
  ]
},
"meta": {
  "methodology": "linear_extrapolation_5yr_rolling_average",
  "emission_source": "EDGAR v2024 (DS-05)",
  "ndc_source": "Climate Watch (DS-06-NDC)",
  "disclaimer": "Projected values are based on simple linear extrapolation and do not constitute scientific forecasts."
}
}

```

Part 5 — 핵심 분석 엔진

5.1 NDC 이행 격차 분석기

이 모듈이 Visual Climate의 가장 강력한 차별화 요소이다.

입력 데이터: **emissions** 테이블(실제 배출) + **ndc_targets** 테이블(목표) + **ndc_content** 테이블(상세 지표)

분석 로직:

```
// lib/analysis/ndc-gap.ts
```

```
interface GapResult {
  onTrack: boolean;
  gapKt: number;           // 양수=초과, 음수=달성 예상
  ambitionGapFactor: number; // 필요 감축속도 / 현재 감축속도
  requiredAnnualReduction: number;
  currentAnnualReduction: number;
  projectedTimeseries: { year: number; value: number }[];
  targetLine: { year: number; value: number }[];
}
```

```
export function calculateNdcGap(
  emissions: { year: number; value: number }[],
  ndc: {
    base_year: number;
    base_year_emissions: number;
    target_year: number;
    reduction_pct: number;
  }
): GapResult {
  // 정렬
  const sorted = [...emissions].sort((a, b) => a.year - b.year);
  const latest = sorted[sorted.length - 1];

  // 최근 5년 평균 연간 변화
  const recentYears = sorted.filter(e => e.year >= latest.year - 5);
  let avgAnnualChange = 0;
  if (recentYears.length >= 2) {
    const changes: number[] = [];
    for (let i = 1; i < recentYears.length; i++) {
      changes.push(
        (recentYears[i].value - recentYears[i - 1].value) /
        (recentYears[i].year - recentYears[i - 1].year)
      );
    }
  }
}
```

```

    }
    avgAnnualChange = changes.reduce((a, b) => a + b, 0) / changes.length;
  }

  // 목표 배출량
  const targetEmissions = ndc.base_year_emissions * (1 - ndc.reduction_pct / 100);

  // 선형 외삽
  const yearsToTarget = ndc.target_year - latest.year;
  const projectedValue = latest.value + avgAnnualChange * yearsToTarget;

  // 격차
  const gap = projectedValue - targetEmissions;

  // 필요 연간 감축량
  const requiredAnnual = (latest.value - targetEmissions) / yearsToTarget;

  // 야망 격차 배수 (현재 속도 대비 필요 속도)
  const ambitionFactor = avgAnnualChange < 0
    ? requiredAnnual / Math.abs(avgAnnualChange)
    : Infinity; // 배출이 증가 중이면 무한대

  // 전망 시계열 생성
  const projected: { year: number; value: number }[] = [];
  for (let y = latest.year; y <= ndc.target_year; y++) {
    projected.push({
      year: y,
      value: Math.max(0, latest.value + avgAnnualChange * (y - latest.year)),
    });
  }

  // 목표 직선
  const targetLine = [
    { year: ndc.base_year, value: ndc.base_year_emissions },
    { year: ndc.target_year, value: targetEmissions },
  ];

  return {
    onTrack: gap <= 0,
    gapKt: Math.round(gap),
    ambitionGapFactor: Math.round(ambitionFactor * 100) / 100,
    requiredAnnualReduction: Math.round(requiredAnnual),
    currentAnnualReduction: Math.round(avgAnnualChange),
    projectedTimeseries: projected,
    targetLine,
  };
}

```

5.2 BTR 투명성 스코어보드 계산

// lib/analysis/btr-scoreboard.ts

```
interface BtrScore {
  country_iso3: string;
  submitted: boolean;
  completeness_score: number; // 0-100
  components: {
    nir: boolean;
    crt: boolean;
    ctf: boolean;
    ndc_tracking: boolean;
    adaptation: boolean;
    finance: boolean;
    redd_plus: boolean | null; // null = 해당 없음
    article6: boolean | null;
  };
  tier: 'comprehensive' | 'substantial' | 'basic' | 'not_submitted';
}

export function calculateBtrScore(btr: any): BtrScore {
  if (!btr || !btr.submitted) {
    return {
      country_iso3: btr?.country_iso3 ?? "",
      submitted: false,
      completeness_score: 0,
      components: {
        nir: false, crt: false, ctf: false, ndc_tracking: false,
        adaptation: false, finance: false, redd_plus: null, article6: null,
      },
      tier: 'not_submitted',
    };
  }

  // 6개 핵심 구성 요소 (가중치 동일)
  const core = [
    btr.has_nir, btr.has_crt, btr.has_ctf,
    btr.has_ndc_tracking, btr.has_adaptation, btr.has_finance_support,
  ];
  const coreScore = core.filter(Boolean).length / core.length;

  // 선택 구성 요소 (보너스)
  const optional = [btr.has_redd_plus, btr.has_article6].filter(v => v === true);
  const bonus = optional.length * 5; // 각 5점 보너스

  const score = Math.min(100, Math.round(coreScore * 90 + bonus));
}
```

```

const tier = score >= 80 ? 'comprehensive'
  : score >= 50 ? 'substantial'
  : 'basic';

return {
  country_iso3: btr.country_iso3,
  submitted: true,
  completeness_score: score,
  components: {
    nir: btr.has_nir ?? false,
    crt: btr.has_crt ?? false,
    ctf: btr.has_ctf ?? false,
    ndc_tracking: btr.has_ndc_tracking ?? false,
    adaptation: btr.has_adaptation ?? false,
    finance: btr.has_finance_support ?? false,
    redd_plus: btr.has_redd_plus,
    article6: btr.has_article6,
  },
  tier,
};
}

```

5.3 적응 삼각 분석

// lib/analysis/adaptation-triangle.ts

```

interface TrianglePoint {
  iso3: string;
  name: string;
  x: number; // vulnerability (0-1)
  y: number; // finance per capita (USD, log scale)
  size: number; // inform risk (0-10)
  color: string; // income group → color
  finance_gap: boolean;
}

export function calculateTriangle(
  rows: any[] // v_adaptation_triangle 뷰 결과
): TrianglePoint[] {
  return rows.map(r => ({
    iso3: r.iso3,
    name: r.name_en,
    x: r.ndgain_vulnerability ?? 0,
    y: Math.max(0.1, r.finance_per_capita ?? 0), // log scale이므로 0 방지
    size: r.inform_risk ?? 1,
    color: incomeColor(r.income_group),
    finance_gap: r.finance_gap_flag ?? false,
  }));
}

```

```

    }));
  }

function incomeColor(group: string | null): string {
  switch (group) {
    case 'Low income': return '#CC3311';
    case 'Lower middle income': return '#EE7733';
    case 'Upper middle income': return '#0077BB';
    case 'High income': return '#009988';
    default: return '#BBBBBB';
  }
}

```

Part 6 — 프론트엔드 & 시각화

6.1 국가 기후 대시보드 (핵심 페이지)

이 페이지가 Visual Climate의 전부이다. 한 화면에서 국가의 기후 현황 7개 축을 모두 볼 수 있다.

```
// app/[locale]/country/[iso3]/page.tsx
```

```

import { createClient } from '@lib/supabase/server';
import { getTranslations } from 'next-intl/server';
import { calculateNdcGap } from '@lib/analysis/ndc-gap';
import { calculateBtrScore } from '@lib/analysis/btr-scoreboard';

// Components
import { EmissionsMultiSourceChart } from
'@/components/charts/EmissionsMultiSourceChart';
import { NdcGapChart } from '@/components/charts/NdcGapChart';
import { BtrChecklist } from '@/components/BtrChecklist';
import { VulnerabilityRadar } from '@/components/charts/VulnerabilityRadar';
import { FinanceSummary } from '@/components/FinanceSummary';
import { DisasterTimeline } from '@/components/charts/DisasterTimeline';
import { ReductionPotentialBar } from '@/components/charts/ReductionPotentialBar';
import { ProjectionFan } from '@/components/charts/ProjectionFan';
import { SourceBadge } from '@/components/SourceBadge';

export const revalidate = 86400; // 24h ISR

export default async function CountryDashboard({ params }: { params: Promise<{ iso3: string
}> }) {

```

```

const { iso3 } = await params;
const t = await getTranslations('Dashboard');
const supabase = await createClient();

// 통합 데이터 한 번에 가져오기
const { data: dashboard } = await supabase.rpc('fn_country_dashboard', { p_iso3: iso3 });
if (!dashboard?.country) return <div>Country not found</div>;

const d = dashboard;

// NDC 격차 계산 (클라이언트 사이드가 아닌 서버에서)
const gapResult = d.ndc && d.emissions_timeseries
  ? calculateNdcGap(
      d.emissions_timeseries.filter((e: any) => e.gas === 'GHG_total' && e.source_id ===
'DS-05'),
      d.ndc
    )
  : null;

// BTR 스코어 계산
const btrScore = calculateBtrScore(d.btr);

return (
  <div className="max-w-7xl mx-auto px-4 py-8 space-y-8">
    { /* — 헤더 — */ }
    <header>
      <h1 className="text-3xl font-bold">{d.country.name_en}</h1>
      <div className="flex flex-wrap gap-2 mt-2 text-sm">
        <span className="bg-slate-100 px-2 py-0.5 rounded">{d.country.region_wb}</span>
        <span className="bg-slate-100 px-2 py-0.5
rounded">{d.country.income_group}</span>
        {d.country.is_sids && <span className="bg-blue-100 px-2 py-0.5
rounded">SIDS</span>}
        {d.country.is_ldc && <span className="bg-amber-100 px-2 py-0.5
rounded">LDC</span>}
        {d.country.is_annex1 && <span className="bg-green-100 px-2 py-0.5
rounded">Annex I</span>}
        {d.country.unfccc_group && <span className="bg-purple-100 px-2 py-0.5
rounded">{d.country.unfccc_group}</span>}
      </div>
    </header>

    { /* — 요약 카드 그리드 — */ }
    <section className="grid grid-cols-2 md:grid-cols-4 lg:grid-cols-7 gap-3">
      <SummaryCard label={t('latestCO2')}
value={formatKt(getLatestTotal(d.emissions_latest, 'CO2'))} />
      <SummaryCard label={t('ndcTarget')} value={d.ndc?.reduction_pct ?
`$${d.ndc.reduction_pct}%` : 'N/A'} sub={`by ${d.ndc?.target_year ?? '-'}`} />
    </section>
  </div>
)

```

```

    <SummaryCard label={t('ndcOnTrack')} value={gapResult?.onTrack ? '✓ On track' : '✗
Off track'} color={gapResult?.onTrack ? 'green' : 'red'} />
    <SummaryCard label={t('btrStatus')} value={btrScore.submitted ?
`${btrScore.completeness_score}%` : 'Not submitted'} sub={btrScore.tier} />
    <SummaryCard label={t('ndgain')} value={d.vulnerability?.find((v: any) => v.indicator
=== 'ndgain_index')?.value?.toFixed(1) ?? 'N/A'} />
    <SummaryCard label={t('climateFinance5yr')}
value={formatUsd(d.finance_5yr?.total_usd)} />
    <SummaryCard label={t('disasters10yr')} value={` ${d.disasters_10yr?.count ?? 0}
events`} />
  </section>

```

```

{ /* — 1. 배출 프로파일 (복수 소스) — */ }

```

```

<section>
  <h2 className="text-xl font-semibold mb-4">{t('emissionsProfile')}</h2>
  <EmissionsMultiSourceChart data={d.emissions_timeseries} />
  <SourceBadgeList data={d.emissions_latest} />
</section>

```

```

{ /* — 2. NDC 이행 격차 ★ — */ }

```

```

{gapResult && (
  <section>
    <h2 className="text-xl font-semibold mb-4">{t('ndcGapAnalysis')}</h2>
    <NdcGapChart
      historical={d.emissions_timeseries?.filter((e: any) => e.gas === 'GHG_total' &&
e.source_id === 'DS-05')}
      projected={gapResult.projectedTimeseries}
      targetLine={gapResult.targetLine}
      metadata={{
        gapKt: gapResult.gapKt,
        ambitionFactor: gapResult.ambitionGapFactor,
        requiredAnnual: gapResult.requiredAnnualReduction,
        currentAnnual: gapResult.currentAnnualReduction,
      }}
    />
    <p className="text-sm text-slate-500 mt-2">
      {t('gapDisclaimer')}
    </p>
  </section>
)}

```

```

{ /* — 3. BTR 투명성 체크리스트 ★ — */ }

```

```

<section>
  <h2 className="text-xl font-semibold mb-4">{t('btrTransparency')}</h2>
  <BtrChecklist score={btrScore} />
</section>

```

```

{ /* — 4. 취약성 & 적응 — */ }

```

```

<section className="grid md:grid-cols-2 gap-6">
  <div>
    <h2 className="text-xl font-semibold mb-4">{t('vulnerability')}</h2>
    <VulnerabilityRadar data={d.vulnerability} />
  </div>
  <div>
    <h2 className="text-xl font-semibold mb-4">{t('adaptationPlan')}</h2>
    <NapStatus nap={d.nap} />
  </div>
</section>

{/* — 5. 기후 자원 — */}
<section>
  <h2 className="text-xl font-semibold mb-4">{t('climateFinance')}</h2>
  <FinanceSummary data={d.finance_5yr} />
</section>

{/* — 6. 감축 잠재량 ★ — */}
{d.top_reduction_strategies && (
  <section>
    <h2 className="text-xl font-semibold mb-4">{t('reductionPotential')}</h2>
    <ReductionPotentialBar strategies={d.top_reduction_strategies} />
  </section>
)}

{/* — 7. 기후 전망 — */}
{d.temp_projection && (
  <section>
    <h2 className="text-xl font-semibold mb-4">{t('climateProjection')}</h2>
    <ProjectionFan data={d.temp_projection} />
  </section>
)}

{/* — 8. 재해 타임라인 — */}
<section>
  <h2 className="text-xl font-semibold mb-4">{t('disasterHistory')}</h2>
  <DisasterTimeline countryIso3={iso3} />
</section>
</div>
);
}

```

6.2 NDC Gap Chart (차별화 시각화)

```

// components/charts/NdcGapChart.tsx
'use client';

```

```

import {
  ComposedChart, Area, Line, XAxis, YAxis, CartesianGrid,
  Tooltip, Legend, ResponsiveContainer, ReferenceLine,
} from 'recharts';

interface Props {
  historical: { year: number; value: number }[];
  projected: { year: number; value: number }[];
  targetLine: { year: number; value: number }[];
  metadata: {
    gapKt: number;
    ambitionFactor: number;
    requiredAnnual: number;
    currentAnnual: number;
  };
};

export function NdcGapChart({ historical, projected, targetLine, metadata }: Props) {
  // 데이터 통합: 연도별로 historical, projected, target 합치기
  const yearMap = new Map<number, any>();

  for (const h of historical) {
    yearMap.set(h.year, { year: h.year, actual: h.value });
  }
  for (const p of projected) {
    const existing = yearMap.get(p.year) || { year: p.year };
    existing.projected = p.value;
    yearMap.set(p.year, existing);
  }
  // target은 두 점(base, target)이므로 보간
  if (targetLine.length === 2) {
    const [start, end] = targetLine;
    for (let y = start.year; y <= end.year; y++) {
      const existing = yearMap.get(y) || { year: y };
      existing.target = start.value + (end.value - start.value)
        * ((y - start.year) / (end.year - start.year));
      yearMap.set(y, existing);
    }
  }

  const data = Array.from(yearMap.values()).sort((a, b) => a.year - b.year);
  const isOnTrack = metadata.gapKt <= 0;

  return (
    <div>
      { /* 격차 요약 배너 */ }
      <div className={`rounded-lg p-4 mb-4 ${isOnTrack ? 'bg-green-50 border-green-200' :
        'bg-red-50 border-red-200'} border`} >

```

```

    <p className="font-semibold">
      {isOnTrack
        ? `NDC 목표 달성 궤도에 있습니다.`
        : `현재 추세로는 NDC 목표 대비 ${Math.abs(metadata.gapKt).toLocaleString()} kt
CO2eq 초과 예상`}
    </p>
    <p className="text-sm mt-1">
      필요 연간 감축: {Math.abs(metadata.requiredAnnual).toLocaleString()} kt/yr ·
      현재 감축 속도: {Math.abs(metadata.currentAnnual).toLocaleString()} kt/yr ·
      야망 격차 배수: {metadata.ambitionFactor}x
    </p>
  </div>

  <ResponsiveContainer width="100%" height={450}>
    <ComposedChart data={data}>
      <CartesianGrid strokeDasharray="3 3" />
      <XAxis dataKey="year" />
      <YAxis label={{ value: 'kt CO2eq', angle: -90, position: 'insideLeft' }} />
      <Tooltip />
      <Legend />

      {/* 격차 영역 (projected와 target 사이) */}
      <Area dataKey="projected" fill="#FEE2E2" stroke="none" name="Projected (gap
area)" />

      {/* 실제 배출 */}
      <Line dataKey="actual" stroke="#1E40AF" strokeWidth={2.5}
        name="Actual emissions" dot={{ r: 2 }} />

      {/* 선형 외삽 */}
      <Line dataKey="projected" stroke="#EF4444" strokeWidth={2}
        strokeDasharray="8 4" name="Current trend (projected)" />

      {/* NDC 목표선 */}
      <Line dataKey="target" stroke="#059669" strokeWidth={2.5}
        name="NDC target pathway" />
    </ComposedChart>
  </ResponsiveContainer>
</div>
);
}

```

6.3 BTR 체크리스트 컴포넌트

```

// components/BtrChecklist.tsx
'use client';

```

```

interface BtrScore {
  submitted: boolean;
  completeness_score: number;
  components: Record<string, boolean | null>;
  tier: string;
}

const LABELS: Record<string, string> = {
  nir: 'National Inventory Report (NIR)',
  crt: 'Common Reporting Tables (CRT)',
  ctf: 'Common Tabular Formats (CTF)',
  ndc_tracking: 'NDC Progress Tracking',
  adaptation: 'Adaptation Chapter',
  finance: 'Finance & Support Info',
  redd_plus: 'REDD+ Annex',
  article6: 'Article 6 Cooperation',
};

export function BtrChecklist({ score }: { score: BtrScore }) {
  if (!score.submitted) {
    return (
      <div className="bg-amber-50 border border-amber-200 rounded-lg p-6">
        <p className="font-semibold text-amber-800">BTR1 not yet submitted</p>
        <p className="text-sm text-amber-600 mt-1">
          The first Biennial Transparency Report was due by 31 December 2024.
          101 out of 194 Parties have submitted as of the 2025 synthesis.
        </p>
      </div>
    );
  }

  return (
    <div className="bg-white border rounded-lg p-6">
      <div className="flex items-center justify-between mb-4">
        <div>
          <span className="text-2xl font-bold">{score.completeness_score}%</span>
          <span className="ml-2 text-sm text-slate-500">completeness</span>
        </div>
        <span className={`px-3 py-1 rounded-full text-sm font-medium
          ${score.tier === 'comprehensive' ? 'bg-green-100 text-green-800' :
            score.tier === 'substantial' ? 'bg-blue-100 text-blue-800' :
              'bg-amber-100 text-amber-800'}`}>
          {score.tier}
        </span>
      </div>
      <div className="grid grid-cols-2 gap-2">
        {Object.entries(score.components).map(([key, val]) => (
          <div key={key} className="flex items-center gap-2 text-sm">

```

```

    {val === null
      ? <span className="text-slate-300">—</span>
      : val
      ? <span className="text-green-600 font-bold">✓</span>
      : <span className="text-red-500 font-bold">✗</span>
    }
    <span>{LABELS[key] ?? key}</span>
  </div>
))}
</div>
</div>
);
}

```

6.4 적응 삼각 산점도 (D3.js)

```

// components/charts/AdaptationTriangleScatter.tsx
'use client';

```

```

import { useEffect, useRef } from 'react';
import * as d3 from 'd3';

```

```

interface Point {
  iso3: string; name: string;
  x: number; y: number; size: number;
  color: string; finance_gap: boolean;
}

```

```

export function AdaptationTriangleScatter({ data }: { data: Point[] }) {
  const svgRef = useRef<SVGSVGElement>(null);

```

```

  useEffect(() => {
    if (!svgRef.current || !data.length) return;

```

```

    const W = 800, H = 500, M = { top: 30, right: 30, bottom: 60, left: 80 };
    const w = W - M.left - M.right, h = H - M.top - M.bottom;

```

```

    const svg = d3.select(svgRef.current).attr('viewBox', `0 0 ${W} ${H}`);
    svg.selectAll('*').remove();

```

```

    const g = svg.append('g').attr('transform', `translate(${M.left},${M.top})`);

```

```

    const xScale = d3.scaleLinear().domain([0, 1]).range([0, w]);
    const yScale = d3.scaleLog().domain([0.1, 10000]).range([h, 0]);
    const sizeScale = d3.scaleSqrt().domain([0, 10]).range([3, 20]);

```

```

    // Axes

```

```

g.append('g').attr('transform', `translate(0,${h})`).call(d3.axisBottom(xScale));
g.append('g').call(d3.axisLeft(yScale).ticks(5, '$.0f'));

// Labels
g.append('text').attr('x', w / 2).attr('y', h + 45).attr('text-anchor', 'middle')
  .text('Vulnerability (ND-GAIN) →');
g.append('text').attr('transform', 'rotate(-90)').attr('x', -h / 2).attr('y', -60)
  .attr('text-anchor', 'middle').text('Climate Finance per Capita (USD, log) →');

// Quadrant annotation
g.append('rect').attr('x', w * 0.5).attr('y', yScale(10)).attr('width', w * 0.5)
  .attr('height', h - yScale(10)).attr('fill', '#FEE2E2').attr('opacity', 0.3);
g.append('text').attr('x', w * 0.75).attr('y', h - 10)
  .attr('text-anchor', 'middle').attr('fill', '#991B1B').attr('font-size', 11)
  .text('HIGH VULNERABILITY · LOW FINANCE');

// Dots
const dots = g.selectAll('circle').data(data).enter().append('circle')
  .attr('cx', d => xScale(d.x))
  .attr('cy', d => yScale(Math.max(0.1, d.y)))
  .attr('r', d => sizeScale(d.size))
  .attr('fill', d => d.color)
  .attr('stroke', d => d.finance_gap ? '#991B1B' : 'none')
  .attr('stroke-width', d => d.finance_gap ? 2 : 0)
  .attr('opacity', 0.7);

// Tooltip
const tooltip = d3.select('body').append('div')
  .style('position', 'absolute').style('visibility', 'hidden')
  .style('background', 'white').style('border', '1px solid #ccc')
  .style('padding', '8px').style('border-radius', '4px')
  .style('font-size', '12px');

dots.on('mouseover', (event, d) => {
  tooltip.style('visibility', 'visible')
  .html(`<strong>${d.name}</strong><br/>
    Vulnerability: ${d.x.toFixed(2)}<br/>
    Finance/capita: $$${d.y.toFixed(1)}<br/>
    INFORM Risk: ${d.size.toFixed(1)}`);
})
.on('mousemove', (event) => {
  tooltip.style('top', `${event.pageY - 10}px`)
  .style('left', `${event.pageX + 10}px`);
})
.on('mouseout', () => tooltip.style('visibility', 'hidden'));

return () => tooltip.remove();
}, [data]);

```

```
return <svg ref={svgRef} className="w-full" />;
}
```

Part 7 — PWA · i18n · 접근성

7.1 PWA (Serwist)

// next.config.ts (핵심 부분만)

```
import withSerwistInit from '@serwist/next';
```

```
const withSerwist = withSerwistInit({
  additionalPrecacheEntries: [{ url: '/~offline', revision: gitRevision }],
  swSrc: 'app/sw.ts',
  swDest: 'public/sw.js',
});
```

서비스 워커는 Part 1의 이전 버전과 동일한 구조를 사용한다. 추가로 `/api/v1/country-dashboard?country=*` 패턴을 `StaleWhileRevalidate`로 캐싱하여, 한 번 방문한 국가는 오프라인에서도 볼 수 있다.

7.2 i18n (6개 UN 공식 언어)

// i18n/routing.ts

```
import { defineRouting } from 'next-intl/routing';
export const routing = defineRouting({
  locales: ['en', 'fr', 'es', 'ar', 'zh', 'ru'],
  defaultLocale: 'en',
  localePrefix: 'as-needed',
});
```

// middleware.ts

```
import createMiddleware from 'next-intl/middleware';
import { routing } from './i18n/routing';
export default createMiddleware(routing);
export const config = { matcher: ['/^(en|fr|es|ar|zh|ru)/:path*'] };
```

아랍어(ar)는 RTL 레이아웃을 적용한다. `<html lang={locale} dir={locale === 'ar' ? 'rtl' : 'ltr'}>` 설정과 Tailwind의 `rtl: variant`를 활용한다.

번역 파일(messages/*.json)은 데이터 레이블까지 포함한다. 예: 가스명(CO2 → CO₂ → ثاني أكسيد الكربون), 색터명, 지표명 등.

7.3 접근성 (WCAG 2.1 AA)

모든 차트에 `aria-label`과 대체 텍스트 테이블을 제공한다. 색맹 친화 팔레트(Wong 2011)를 기본으로 사용한다. 키보드 네비게이션으로 차트 데이터 포인트 탐색이 가능하다. 대비율 최소 4.5:1을 보장한다.

Part 8 — 보안 · 테스트 · 배포

8.1 보안

위협	대응
SQL Injection	Supabase SDK parameterized query
XSS	CSP 헤더 + Next.js auto-escape
CSRF	Server Actions origin 검증
IDOR	RLS (공개 읽기, 쓰기는 service role만)
Rate Limit	Vercel Edge Middleware IP 기반 100 req/min
Prompt Injection	시스템 프롬프트 격리, 입력 필터링
토큰 탈취	HTTP-only cookie, 1h JWT 만료, refresh rotation
데이터 무결성	SHA-256 체크섬, S3 원시 파일 보존

8.2 테스트 (AT-01 ~ AT-14)

ID	시나리오	검증
AT-01	국가 대시보드	KOR → 7개 섹션 모두 렌더, 복수 소스 표시
AT-02	NDC Gap ★	KOR → 격차 차트: 실제 배출선 + 외삽선 + 목표선 + 격차 배너
AT-03	BTR 체크리스트 ★	KOR → BTR1 제출 여부 + 구성 요소 체크 + 완전성 점수
AT-04	적응 삼각 ★	글로벌 산점도 → 고취약·저자원 사분면에 LDC/SIDS 분포 확인

AT-0 5	감축 잠재량 ★	KOR → Climate TRACE ERS 기반 상위 5개 전략 막대 그래프
AT-0 6	복수 소스 비교	KOR CO ₂ → WDI vs EDGAR vs CT 세 선 병렬, 차이 설명
AT-0 7	국가 비교	KOR vs JPN vs CHN → 동일 소스 비교
AT-0 8	오프라인	이전 방문 국가 캐시 로드, “offline” 표시
AT-0 9	다국어	en→fr→ar → UI 레이블 + RTL + 숫자 형식
AT-1 0	ETL 실행	DS-01 수동 → etl_logs success, quarantine 0
AT-11	성능	Lighthouse ≥ 90, LCP ≤ 2.5s
AT-1 2	접근성	axe-core 0 violations, 차트 대체 텍스트
AT-1 3	보안	CRON_SECRET 없이 ETL → 401
AT-1 4	AI 챗	“한국 NDC 달성 가능?” → 출처 인용 + gap 수치 포함 응답

8.3 배포 전략

환경: **main** → Production (vercel.app), **develop** → Preview, PR → Preview per-commit.

CI/CD (GitHub Actions):

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci
      - run: npx supabase start # Local Supabase
      - run: npx vitest --coverage
      - run: npx playwright install --with-deps
```

- run: npx playwright test
- run: npx lighthouse-ci autorun

8.4 구현 일정 (20주)

주	마일스톤
1-2	프로젝트 셋업, DB 마이그레이션, Vercel/Supabase 연결, CI
3-5	ETL 프레임워크 + DS-01, DS-05, DS-02, DS-06-NDC, DS-BTR
6-8	나머지 ETL (DS-03,04,09,11,14,15,18,27), 데이터 검증
9-10	핵심 API (/emissions, /ndc, /btr, /gap-analysis, /country-dashboard)
11-13	국가 대시보드 페이지 + NDC Gap Chart + BTR Checklist
14-15	적응 삼각, 감축 잠재량, 기후 전망, 재해 타임라인 시각화
16-17	i18n (6개 언어), PWA, 오프라인, 접근성
18-19	AI 챗, 테스트 (unit + E2E + Lighthouse), 성능 튜닝
20	베타 테스트, 버그 수정, 프로덕션 릴리스

부록 A — 전체 데이터 소스 카탈로그

ID	소스명	패턴	대상 테이블	주 기	레코드(추정)	라이선스	API/URL
DS-01	World Bank WDI	A	emissions, climate_indicators	반기	66K	CC BY 4.0	api.worldbank.org/v2/
DS-02	Climate TRACE	A+B	emissions, reduction_potential	분기	50K+2.7M	CC BY 4.0	downloads.climate TRACE.org
DS-03	EM-DAT	B	disasters	분기	27K	비상업 무료	public.emdat.be
DS-04	ND-GAIN	B	vulnerability	연간	15K	CC BY 4.0	gain.nd.edu

DS-05	EDGAR v2024	B	emissions	연간	800K	JRC/IEA	edgar.jrc.ec.europa.eu
DS-06	Climate Watch	A	emissions, ndc_content	연간	120K+50K NDC	CC BY 4.0	climatewatchdata.org/api/v1/
DS-07	IEA WEO	B	emissions	연간	25K	유료	iea.org
DS-08	UNFCCC NDC	D	ndc_targets	비정기	2K	Public	unfccc.int/NDCREG
DS-09	GISTEMP v4	B	climate_indicators	월간	26K	NASA	data.giss.nasa.gov
DS-10	Copernicus C3S	A	climate_indicators	월간	1K	Copernicus	cds.climate.copernicus.eu
DS-11	FAOSTAT	B	emissions	연간	500K	CC BY-NC-SA	fao.org/faostat
DS-12	IRENA	B	climate_indicators	연간	80K	Public	irena.org/Data
DS-13	GCB 2024	B	emissions	연간	15K	CC BY 4.0	globalcarbonbudget.org
DS-14	INFORM Risk	A	vulnerability	연간	30K	Public	INFORM API
DS-15	GCF	A	climate_finance	분기	2K	Public	developer.gcfund.org
DS-16	IIASA AR6	B	climate_indicators	비정기	5M	CC BY 4.0	data.ene.iiasa.ac.at/ar6/
DS-17	UN SDG	A	climate_indicators	연간	10K	Public	unstats.un.org/sdg-api/
DS-18	CCKP	A	country_projections	비정기	200K	WB Terms	cckpapi.worldbank.org
DS-19	NOAA CO ₂	B	climate_indicators	월간	800	Public Domain	gml.noaa.gov

DS-2 0	NASA Sea Level	B	climate_indica tors	월 간	400	NASA	podaac.jpl.nasa.gov
DS-2 1	NSIDC Sea Ice	B	climate_indica tors	월 간	550	Public	nsidc.org
DS-2 2	UNEP Adapt Gap	D	nap_status	연 간	200	Public	unep.org
DS-2 3	CPI Climate Finance	B	climate_finan ce	연 간	5K	CC BY	climatepolicyiniti ative.org
DS-2 4	WRI Aqueduct	A	vulnerability	연 간	50K	CC BY 4.0	wri.org/aqueduct
DS-2 5	Global Energy Monitor	B	climate_indica tors	분 기	40K	CC BY 4.0	globalenergymonito r.org
DS-2 6	Carbon Brief	D	ndc_content	비 정 기	60	CC BY-NC- ND	carbonbrief.org
DS-2 7	Climate Action Tracker	A	ndc_targets	분 기	400	CC BY	climateactiontrack er.org
DS-2 8	OECD DAC	A	climate_finan ce	연 간	10K	OECD Terms	stats.oecd.org
DS-2 9	IMF Climate	A	climate_indica tors	연 간	15K	IMF Terms	climatedata.imf.or g
DS-3 0	WB Sovereig n Risk	A	vulnerability	연 간	5K	WB Terms	worldbank.org
DS-3 1	WHO Climate Health	B	vulnerability	연 간	20K	Public	who.int
DS-3 2	IDMC Displace ment	A	disasters	연 간	8K	CC BY-NC	idmc.int

DS-33	Munich Re NatCat	D	disasters	연간	1K	Restricted	munichre.com
DS-34	Swiss Re Sigma	D	disasters	연간	500	Restricted	swissre.com
DS-35	Our World in Data	B	emissions, climate_indicators	연간	300K	CC BY	github.com/owid
DS-36	Global Forest Watch	A	climate_indicators	월간	100K	CC BY 4.0	globalforestwatch.org
DS-37	Germanwatch CRI	D	vulnerability	연간	200	출처표기	germanwatch.org
DS-38	UNEP Emissions Gap	D	ndc_content	연간	100	Public	unep.org
DS-39	IEA CO ₂ (Free)	B	emissions	연간	15K	IEA Free	iea.org
DS-40	UNFCCC GHG (CRF)	A	emissions	연간	200K	Public	di.unfccc.int
DS-BTR	UNFCCC BTR Status	B+D	btr_status	반기	200	Public	unfccc.int/first-biennial-transparency-reports
DS-NAP	NAPcentral	B+D	nap_status	연간	150	Public	napcentral.org

총 추정 레코드: **~9.5M** (IIASA AR6 5M 포함, 제외 시 ~4.5M)

이것이 기술 사양서 전체입니다. 지금 출력된 분량이 실제 페이지 기준 약 **45-50**페이지에 해당합니다.

다른 플랫폼에 없는 **Visual Climate**만의 차별화 포인트가 기술 전반에 녹아 있습니다: NDC 이행 격차 분석기([fn_ndc_gap_analysis](#), [NdcGapChart](#)), BTR 투명성 스코어보드([btr_status](#) 테이블, [BtrChecklist](#)), 적응-취약성-재원 삼각 분석([v_adaptation_triangle](#), [AdaptationTriangleScatter](#)), 감축 잠재량 매핑(Climate TRACE ERS → [reduction_potential](#)), 국가별 7축 통합 대시보드([fn_country_dashboard](#)).

수정이나 추가할 섹션 말씀해 주세요.